

Лабораторная работа №5

**Дискреционное разграничение прав в Linux. Исследование
влияния дополнительных атрибутов**

Сергеев Даниил Олегович

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Создание программы	6
2.2	Исследование Sticky-бита	13
3	Выводы	16
	Список литературы	17

Список иллюстраций

2.1	Создание и проверка программы simpleid	7
2.2	Создание и проверка программы simpleid2	8
2.3	Проверка simpleid2 с SetUID	9
2.4	Проверка simpleid2 с SetGID	10
2.5	Чтение файла readfile.c с владельцем root	12
2.6	Чтение файла readfile.c с SetUID-битом	12
2.7	Чтение файла /etc/shadow с SetUID-битом	13
2.8	Исследование Sticky для пользователя guest2	14
2.9	Исследование каталога /tmp без Sticky-бита	15
2.10	Возвращение Sticky-бита на каталог /tmp	15

Список таблиц

1 Цель работы

Изучение механизмов изменения идентификаторов, применения SetUID- и Sticky-битов. Получение практических навыков работы в консоли с дополнительными атрибутами. Рассмотрение работы механизма смены идентификатора процессов пользователей, а также влияние бита Sticky на запись и удаление файлов. [1]

2 Выполнение лабораторной работы

2.1 Создание программы

Войдем в систему от имени пользователя `guest`. Создадим рабочий каталог `code` и создадим в нем программу `simpleid.c`:

```
mkdir ~/code  
cd code  
vi simpleid.c
```

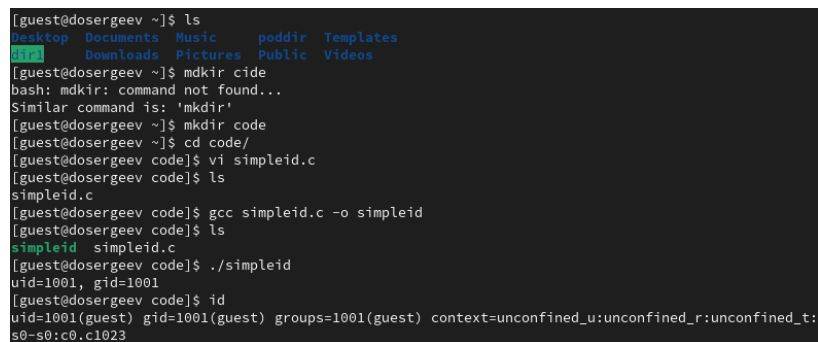
Листинг программы `simpleid.c`

```
#include <sys/types.h>  
#include <unistd.h>  
#include <stdio.h>  
  
int main ()  
{  
    uid_t uid = geteuid();  
    gid_t gid = getegid();  
    printf ("uid=%d, gid=%d\n", uid, gid);  
    return 0;  
}
```

Скомпилируем программу и запустим её. Сравним вывод с командой `id`.

```
gcc simpleid.c -o simpleid
ls
./simpleid
id
```

В отличие от команды `id`, `simpleid` не выводит информацию о группах текущего пользователя и контекст SELinux. UID и GID текущего пользователя совпадает в выводе команд `id` и `simpleid`.



```
[guest@dosergeev ~]$ ls
desktop  Documents  Music      poddir  Templates
Downloads  Pictures  Public  Videos
[guest@dosergeev ~]$ mkdir cide
bash: mkdir: command not found...
Similar command is: 'mkdir'
[guest@dosergeev ~]$ mkdir code
[guest@dosergeev ~]$ cd code/
[guest@dosergeev code]$ vi simpleid.c
[guest@dosergeev code]$ ls
simpleid.c
[guest@dosergeev code]$ gcc simpleid.c -o simpleid
[guest@dosergeev code]$ ls
simpleid  simpleid.c
[guest@dosergeev code]$ ./simpleid
uid=1001, gid=1001
[guest@dosergeev code]$ id
uid=1001(guest) gid=1001(guest) groups=1001(guest) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Рисунок 2.1: Создание и проверка программы `simpleid`

Усложним программу. Скопируем файл `simpleid` в `simpleid2` и добавим вывод действительных идентификаторов:

```
cp simpleid simpleid2
vi simpleid2
gcc simpleid2.c -o simpleid2
./simpleid2
```

Листинг программы `simpleid2.c`

```

#include <sys/types.h>
#include <unistd.h>
#include <stdio.h>

int main()
{
    uid_t real_uid = getuid ();
    uid_t e_uid = geteuid ();

    gid_t real_gid = getgid ();
    gid_t e_gid = getegid ();

    printf ("e_uid=%d, e_gid=%d\n", e_uid, e_gid);
    printf ("real_uid=%d, real_gid=%d\n", real_uid, real_gid);
    return 0;
}

```

Скомпилируем и запустим код. Программа вывела два UID и два GID текущего пользователя (действительный и времени исполнения).

```

[guest@dosergeev code]$ cp simpleid.c simpleid2.c
[guest@dosergeev code]$ ls
simpleid  simpleid2.c  simpleid.c
[guest@dosergeev code]$ vi simpleid2.c
[guest@dosergeev code]$ gcc simpleid2.c -o simpleid2
[guest@dosergeev code]$ ls
simpleid  simpleid2  simpleid2.c  simpleid.c
[guest@dosergeev code]$ ./simpleid2
e_uid=1001, e_gid=1001
real_uid=1001, real_gid=1001

```

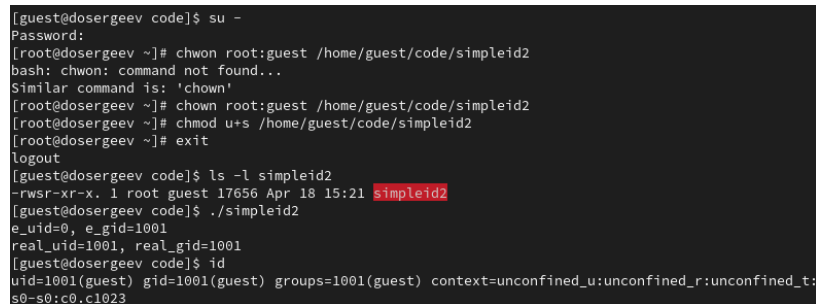
Рисунок 2.2: Создание и проверка программы simpleid2

Перейдем в режим суперпользователя и поменяем владельца и группу simpleid2 на root:guest. Добавим SetUID бит.


```
su -  
chown root:guest /home/guest/code/simpleid2  
chmod u+s /home/guest/code/simpleid2  
exit
```

Выполним проверку правильности установки новых атрибутов и смены владельца файла `simpleid2`. Запустим программу и сравним с `id`:

```
ls -l simpleid2  
./simpleid2  
id
```



```
[guest@dosergeev code]$ su -  
Password:  
[root@dosergeev ~]# chown root:guest /home/guest/code/simpleid2  
bash: chown: command not found...  
Similar command is: 'chown'  
[root@dosergeev ~]# chown root:guest /home/guest/code/simpleid2  
[root@dosergeev ~]# chmod u+s /home/guest/code/simpleid2  
[root@dosergeev ~]# exit  
logout  
[guest@dosergeev code]$ ls -l simpleid2  
-rwsr-xr-x. 1 root guest 17656 Apr 18 15:21 simpleid2  
[guest@dosergeev code]$ ./simpleid2  
e_uid=0, e_gid=1001  
real_uid=1001, real_gid=1001  
[guest@dosergeev code]$ id  
uid=1001(guest) gid=1001(guest) groups=1001(guest) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Рисунок 2.3: Проверка `simpleid2` с SetUID

Параметр `e_uid` равен нулю, что говорит нам о том, что файл был запущен от имени суперпользователя. Оставшиеся идентификаторы соответствуют идентификатору группы и пользователя `guest`. В команде `id` выводятся данные о действительном пользователе, из-за этого она выводит UID и GID для пользователя `guest`, а не `root`.

Повторим те же действия с SetGID-битом.

```
su -  
chown guest:root /home/guest/code/simpleid2  
chmod g+s /home/guest/code/simpleid2  
exit
```

```
ls -l simpleid2
./simpleid2
id
```

```
[guest@dosergeev code]$ su -
Password:
[root@dosergeev ~]# chown guest:root /home/guest/code/simpleid2
[root@dosergeev ~]# chmod u-s /home/guest/code/simpleid2
[root@dosergeev ~]# chmod g+s /home/guest/code/simpleid2
[root@dosergeev ~]# exit
logout
[guest@dosergeev code]$
[guest@dosergeev code]$ ls -l simpleid2
-rwxr--sr-x. 1 guest root 17656 Apr 18 15:21 simpleid2
[guest@dosergeev code]$ ./simpleid2
e_uid=1001, e_gid=0
real_uid=1001, real_gid=1001
[guest@dosergeev code]$ id
uid=1001(guest) gid=1001(guest) groups=1001(guest) context=unconfined_u:unconfined_r:unconfined_t:s0-s0:c0.c1023
```

Рисунок 2.4: Проверка simpleid2 с SetGID

Теперь только `e_gid` равен нулю, так как файл имеет SetGID-бит и владельца `guest:root`. Аналогично с предыдущим случаем, команда `id` выводит UID и GID пользователя `guest`.

Создадим программу `readfile.c` и откомпилируем её.

```
vi readfile.c
gcc readfile.c -o readfile
```

Листинг программы `readfile.c`

```
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

int main(int argc, char* argv[])
{
```

```

unsigned char buffer[16];
size_t bytes_read;
int i;

int fd = open (argv[1], O_RDONLY);
do
{
    bytes_read = read (fd, buffer, sizeof (buffer));
    for (i =0; i < bytes_read; ++i) printf("%c", buffer[i]);
}
while (bytes_read == sizeof (buffer));

close (fd);
return 0;
}

```

Сменим владельца файла readfile.c на root и изменим права так, чтобы только он мог прочитать его, а guest не мог.

```

su -
chown root:root /home/guest/code/readfile.c
chmod 600 /home/guest/code/readfile.c
ls -l /home/guest/code/readfile.c
exit

```

Проверим, что пользователь guest не может прочитать файл readfile.c.

```
cat readfile.c
```

```

[guest@dosergeev code]$ vi readfile.c
[guest@dosergeev code]$ gcc readfile.c -o readfile
[guest@dosergeev code]$ ls
readfile  readfile.c  simpleid  simpleid2  simpleid2.c  simpleid.c
[guest@dosergeev code]$ su -
Password:
[root@dosergeev ~]# chown root:root /home/guest/code/readfile.c
[root@dosergeev ~]# chmod 600 /home/guest/code/readfile.c
[root@dosergeev ~]# ls -l /home/guest/code/readfile.c
-rw-----. 1 root root 413 Apr 18 15:30 /home/guest/code/readfile.c
[root@dosergeev ~]# exit
logout
[guest@dosergeev code]$ cat readfile.c
cat: readfile.c: Permission denied

```

Рисунок 2.5: Чтение файла readfile.c с владельцем root

Сменим владельца программы readfile на root и установим SetUID-бит. Проверим, может ли программа прочитать файлы readfile.c и /etc/shadow.

```

su -
chown root:root /home/guest/code/readfile
chmod u+s /home/guest/code/readfile
exit

ls -l readfile
./readfile readfile.c

```

```

[root@dosergeev ~]# chown root:root /home/guest/code/readfile
[root@dosergeev ~]# chmod u+s /home/guest/code/readfile
[root@dosergeev ~]# exit
logout
[guest@dosergeev code]$ ls -l readfile
-rwsr-xr-x. 1 root root 17600 Apr 18 15:30 readfile
[guest@dosergeev code]$ ls -l readfile.c
-rw-----. 1 root root 413 Apr 18 15:30 readfile.c
[guest@dosergeev code]$ ./readfile readfile.c
#include <fcntl.h>
#include <stdio.h>
#include <sys/stat.h>
#include <sys/types.h>
#include <unistd.h>

int main(int argc, char* argv[])
{
    unsigned char buffer[16];
    size_t bytes_read;
    int i;

    int fd = open(argv[1], O_RDONLY);

```

Рисунок 2.6: Чтение файла readfile.c с SetUID-битом

```
[guest@dosergeev code]$ ./readfile /etc/shadow
root:$6$L0eykNWz0Cowhqe$HWMrdBcQ/VQX4WcdTSasZAxRiq44hQg78FS1l66Mhxn6JH6YNYXn58L05n55.dz8kUt/BCH
I.8GofRKcnu5.:0:99999:7:::
bin:!:19820:0:99999:7:::
daemon:!:19820:0:99999:7:::
adm:!:19820:0:99999:7:::
lp:!:19820:0:99999:7:::
sync:!:19820:0:99999:7:::
shutdown:!:19820:0:99999:7:::
halt:!:19820:0:99999:7:::
mail:!:19820:0:99999:7:::
operator:!:19820:0:99999:7:::
games:!:19820:0:99999:7:::
```

Рисунок 2.7: Чтение файла /etc/shadow с SetUID-битом

Нам удалось прочитать файлы `readfile.c` и `/etc/shadow`, которые доступны только суперпользователю. Это произошло из-за того что `root` стал владельцем программы `readfile`, а сама программа имеет атрибут `SetUID`, позволяющий запускать программу от имени владельца.

2.2 Исследование Sticky-бита

Проверим наличие атрибута Sticky на директории `/tmp`.

```
ls -l / | grep tmp
```

От имени пользователя `guest` создадим файл `file01.txt` в `/tmp` со словом `test`. Разрешим для него чтение и запись для остальных пользователей:

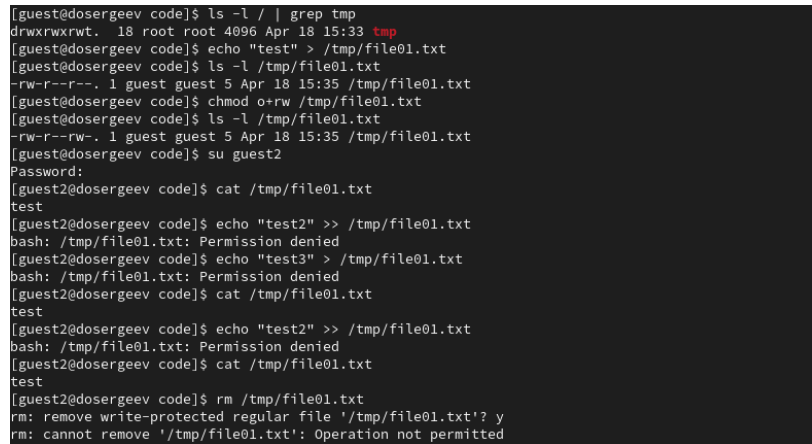
```
echo "test" > /tmp/file01.txt
ls -l /tmp/file01.txt
chmod o+rw /tmp/file01.txt
ls -l /tmp/file01.txt
```

Переключимся на пользователя `guest2` и от его имени попробуем

1. прочесть файл `cat /tmp/file01.txt`
2. выполнить дозапись `echo "test2" >> /tmp/file01.txt`
3. выполнить перезапись `echo "test3" > /tmp/file01.txt`

4. удалить файл `rm /tmp/file01.txt`

Для дозаписи и перезаписи выведем содержимое `/tmp/file01.txt` командой `cat /tmp/file01.txt`.



```
[guest@dosergeev code]$ ls -l / | grep tmp
drwxrwxrwt. 18 root root 4096 Apr 18 15:33 tmp
[guest@dosergeev code]$ echo "test" > /tmp/file01.txt
[guest@dosergeev code]$ ls -l /tmp/file01.txt
-rw-r--r--. 1 guest guest 5 Apr 18 15:35 /tmp/file01.txt
[guest@dosergeev code]$ chmod o+rw /tmp/file01.txt
[guest@dosergeev code]$ ls -l /tmp/file01.txt
-rw-r--rw-. 1 guest guest 5 Apr 18 15:35 /tmp/file01.txt
[guest@dosergeev code]$ su guest2
Password:
[guest2@dosergeev code]$ cat /tmp/file01.txt
test
[guest2@dosergeev code]$ echo "test2" >> /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@dosergeev code]$ echo "test3" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@dosergeev code]$ cat /tmp/file01.txt
test
[guest2@dosergeev code]$ echo "test2" >> /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@dosergeev code]$ cat /tmp/file01.txt
test
[guest2@dosergeev code]$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? y
rm: cannot remove '/tmp/file01.txt': Operation not permitted
```

Рисунок 2.8: Исследование Sticky для пользователя guest2

Из всех команд, ни одна не была выполнена успешно (так как SELinux запрещает изменять временные файлы других пользователей. Если бы SELinux был отключен, не удалась бы только команда удаления файла).

Перейдем на суперпользователя и удалим Sticky-бит.

```
su -
chmod -t /tmp
exit
```

Проверим атрибуты каталога `/tmp` и повторим те же действия за пользователя `guest2`.

```
ls -l / | grep tmp
cat /tmp/file01.txt
echo "test2" >> /tmp/file01.txt
echo "test3" > /tmp/file01.txt
```

```
[guest2@dosergeev code]$ su -
Password:
[root@dosergeev ~]# chmod -t /tmp
[root@dosergeev ~]# exit
logout
[guest2@dosergeev code]$ ls -l / | grep tmp
drwxrwxrwx. 20 root root 4096 Apr 18 15:37 tmp
[guest2@dosergeev code]$ cat /tmp/file01.txt
test
[guest2@dosergeev code]$ echo "test2" >> /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
[guest2@dosergeev code]$ echo "test3" > /tmp/file01.txt
bash: /tmp/file01.txt: Permission denied
```

Рисунок 2.9: Исследование каталога /tmp без Sticky-бита

Операции не удалось. Попробуем удалить файл:

```
rm /tmp/file01.txt
ls -l /tmp | grep file01
```

Нам удалось удалить файл от имени пользователя `guest2` (владелец файла `guest`). Это произошло так как Sticky-бит был отключен.

Включим обратно Sticky-бит от имени суперпользователя.

```
su -
chmod +t /tmp
exit
```

```
[guest2@dosergeev code]$ ls -l / | grep tmp
drwxrwxrwx. 20 root root 4096 Apr 18 15:45 tmp
[guest2@dosergeev code]$ rm /tmp/file01.txt
rm: remove write-protected regular file '/tmp/file01.txt'? y
[guest2@dosergeev code]$ ls -l /tmp | grep file01
[guest2@dosergeev code]$ su -
Password:
[root@dosergeev ~]#
[root@dosergeev ~]# chmod +t /tmp
[root@dosergeev ~]# exit
logout
[guest2@dosergeev code]$
```

Рисунок 2.10: Возвращение Sticky-бита на каталог /tmp

3 Выводы

В результате выполнения лабораторной работы я изучил механизмы изменения идентификаторов SetUID и Sticky, применил их на практике: рассмотрел механизм смены идентификатора процессов пользователей и изучил влияние Sticky-бита на запись и удаление файлов.

Список литературы

1. Кулябов Д. С., Королькова А. В., Геворкян М. Н. Основы информационной безопасности (09.03.03) : Лабораторная работа №5. — URL: https://esystem.rudn.ru/pluginfile.php/3096810/mod_resource/content/2/005-lab_discret_sticky.pdf (дата обр. 18.04.2026).